Agentic AI Project Synopsis
Group 8 | CSE 1

1. Project Title: Redliner - B2B Contract Negotiaitions and Redlining

2. Team Members:

Name	Enrolment No.
Divyam Joshi	230452
Amrit Parashar	230453
Siddharth Jain	230482
Manit Trehan	230489

3. Problem Definition

B2B contract negotiations take a long time and are delayed due to several reasons.

The average B2B contract execution takes 3.4 weeks and manual review and redlining cycles (legal bottlenecks) taking 5-10 days per iteration. Because 71% of organizations still rely on manual contract workflows, the speed of the deal grinds to a halt. [2]

Professional organizations experience average contract value erosion of 8.6% due to contract negotiations. [1] 71% of businesses according to World Commerce & Contracting still use manual processes for contracting. [2]

Redliner is designed for enterprise organizations that handle recurring B2B vendor onboarding and procurement-

Redliner serves procurement managers, in-house legal counsel, and finance leaders looking to streamline vendor onboarding.
Procurement team will use it to handle negotiations back and forth around pricing, payment terms etc. The legal team will offload the contract reviews to automatically generate native Word tracked changes and comments aligned with corporate playbooks.
CFOs can oversee the whole process.
Also, the vendor counterparties benefit from clear and fast feedback delivered directly over email webhooks.
All of these teams combined should leads to lower contraction execution times.

4. Objectives & Expected Outcomes

Objectives:

- Build a cyclic agentic workflow with state persistence that processes B2B negotiations autonomously. This should work asynchronously.

- Develop a deterministic scoring system which combines pricing models, payment terms, and risk penalties while ensuring that the counter proposals maximize our deal value.

- Monitor contract terms against thresholds. Automatically pause execution and send high-risk or over-budget terms to dashboard for human review., or when negotiations do not cross threshholds, send for approval.

Agentic AI Project Synopsis
Group 8 | CSE 1
Expected Outcomes:

- The project will deliver a system that takes vendor proposals and generates fully redlined .docx contracts with native tracked changes, explanatory comments, and structured counter-proposal emails.

- React and TypeScript dashboard that provides procurement, legal, and finance teams with control panel.
  It will display deal summaries and the tradeoffs between risk and price. Approval gates pause the agent execution when deal terms cross thresholds that we set before the deal starts.

- Risk and value: A system designed to improve deal quality and strengthen guardrails by supporting alignment between legal and financial terms. This is intended to reduce the risk that teams inadvertently accept high legal liabilities or unfavourable terms in exchange for nominal price discounts.

5. Proposed Methodology

Agents

1) Main Orchestrator - The supervisor agent is the first node in the main execution graph. It manages state saved in Supabase at asynchronous thread checkpoints and checks whether the overall deal has converged. It assigns work to specialized agents or subgraphs and decides when human approval is needed.

2) Core Agents -
   a) Procurement Negotiator: Review unit pricing, volume discounts, and service-level agreements (SLAs). It sets target prices and plans concessions.

   b) Legal Redliner: Checks contract terms including compensation, liability limits, intellectual property, governing law, and termination, against corporate legal playbooks. It identifies required changes and specifies the exact clause edits.

   c) Finance, Compliance & Risk: the boundary checker. It runs the Python trade-off scoring module to evaluate financial penalties, payment terms (e.g., Net 30 vs. Net 60), and regulatory risk scores against hard corporate limits.

   d) Vendor Intelligence: Conducts due diligence and background checks on counterparties by querying corporate registries, and credit indicators, etc.

   e) Email Writer: Combines negotiation decisions, contract context, and justifications into professional emails for the vendor party.

3) Subgraphs -
   a) Document Parser - This subgraph receives vendor .docx and .pdf attachments through email webhooks. If a file is not accessible or scanned, the OCR Agent reads it using OCR, else we try XML extraction.
   Segmenter Agent creates an indexed array of typed clause objects, such as LIABILITY, IP_RIGHTS etc.

Agentic AI Project Synopsis
Group 8 | CSE 1

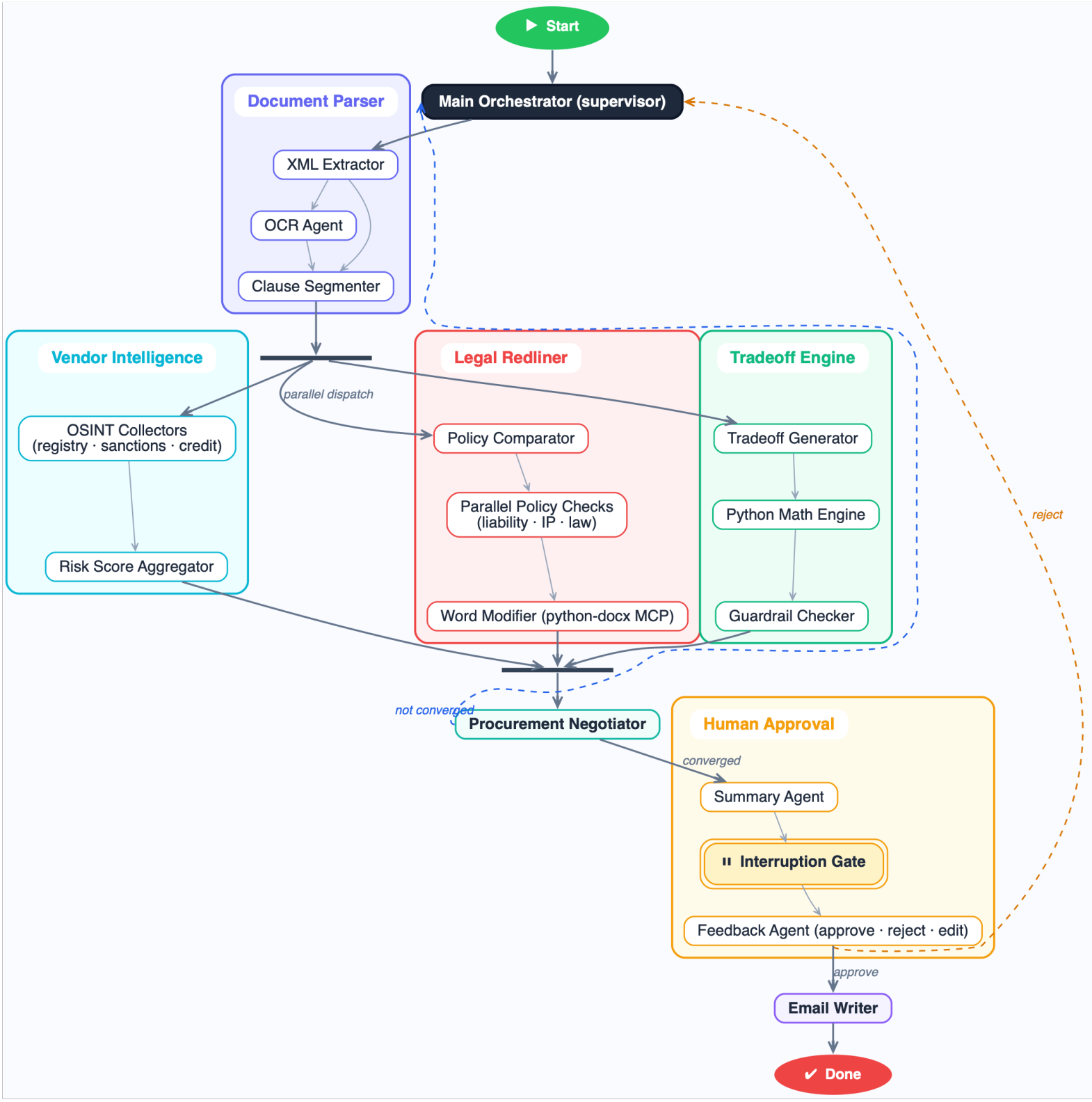
b) Playbook Rule Matcher compares structured clauses with active corporate policies on liability caps, governing law, and IP retention. Parallel Agents check three areas: Compensation & Liability (caps against deal value that we set before the deal starts), Intellectual Property, and Law.
The Redline Action Generator combines the findings into exact strikethroughs, replacements, and comments.
Word Modifier uses python-docx MCP server to edit the native .docx XML.

c) The Tradeoff Generator proposes counter-offers, such as a 5% unit price increase in exchange for moving from Net 30 to Net 60. It sends the proposed terms through MCP to a deterministic scoring module, which calculates a normalized utility score. Guardrail Checker compares each score with absolute budget caps and minimum utility limits.

d) Vendor Information - This subgraph checks counterparties and screens for sanctions. Parallel OSINT Collectors query external APIs through MCP for corporate registry records, sanctions, domain age, and credit indicators. Exception Handler manages timeouts, rate limits, and retries. Score Aggregator combines the external data into a standardized score and saves it to Supabase.

e) Human Approval - This subgraph pauses deals that go beyond our thresholds, prepares data for the dashboard, and applies human decisions. The Summary Agent summarizes proposed redlines, financial trade-offs, and risk scores in plain English. The Interruption Gate pauses execution and saves the state to Supabase. Feedback Agent receives approve, reject, or “Manual Edit,” updates the state, and goes back to the Master Orchestrator.

LLM / Model & Agentic Framework:
LangGraph paired with GPT-6 Luna for high volume, low latency agentic tasks (such as document extraction and clause tagging), with DeepSeek 4 Pro for complex reasoning, multi objective optimization.


Start
Main Orchestrator (supervisor)
Document Parser
XML Extractor
OCR Agent
Clause Segmenter
Vendor Intelligence
OSINT Collectors
(registry · sanctions · credit)
Risk Score Aggregator
parallel dispatch
Legal Redliner
Policy Comparator
Parallel Policy Checks
(liability · IP · law)
Word Modifier (python-docx MCP)
Tradeoff Engine
Tradeoff Generator
Python Math Engine
Guardrail Checker
reject
not converged
Procurement Negotiator
Human Approval
converged
Summary Agent
Interruption Gate
Feedback Agent (approve · reject · edit)
approve
Email Writer
Done

Agentic AI Project Synopsis
Group 8 | CSE 1

6. Feasibility & Resources

Data Source:

APIs/tools

- Python with LangGraph for agent orchestration and backend.
- TypeScript + React for frontend dashboard.
- Mixture of Groq and OpenRouter for embedding and LLM inference.
- Cloudflare Email Routing + Workers for email webhooks.
- Supabase for storing audit logs, vendor risk scores, and other metadata.
- MCP for agent to tool communication (python-docx execution, Cloudflare Email etc.)
- Langfuse free tier for agent observability, pricing etc.

Frameworks:

- LangGraph: Multi-agent orchestration framework for cyclic workflows, sub-graphs, and HITL state management.
- React + Typescript: Frontend web framework for building the interactive negotiation dashboard.
- FastAPI + Pydantic: Python web framework and data validation.
- Cloudflare Workers SDK: Serverless edge runtime for email webhooks and attachment parsing.

Computation

- Using Opencode Go (Rs. 930) and OpenRouter for model inference.
- A VPS may be required to self host Langfuse if free tier does not fulfil our requirements.

7. Team Members & Individual Understanding

Member	Role/Responsibility	Individual Understanding
Divyam Joshi	Frontend + Webhooks	React, Javascript
Amrit Parashar	Document + API and Tools	MCP, APIs, Javascript
Siddharth Jain	Orchestration System	Python and LangGraph
Manit Trehan	Python Module + Evals	Python, Deployment Architecture

8. Project Timeline

Activity	Week(s)	Deliverable
Problem & Design	7 October	Full architecture including subgraphs
Development /Coding	21 October	Core agents, subgraphs an d tools
Integration & Testing/Debugging	28 October	Full Running Pipeline

Activity	Week(s)	Deliverable
Evaluation & Refinement	2 November	Benchmarks and risk validation
Presentation/Interface UI	28 October 2 November	Intial - Oct Final - Nov
Deployment	8 November	Deployed backend, Supabase state and UI
Final Demo	10 November	Demo

9. Team Responsibility Matrix

Activity	Divyam Joshi	Amrit Parashar	Siddharth Jain	Manit Trehan
Agent Design		Main	Main	-
Agent/Tool Development	Support	Main	Main	-
Integration & Testing	-	Main	-	Main
Evaluation & Documentation	Main	-	Support	Main
Presentation/Interface UI	Main	-	Support	-
Deployment	Support	-	-	Main

References

[1] World Commerce & Contracting, & Deloitte. (2023). *The ROI of contracting excellence.* https://www.worldcc.com/Portals/IACCM/Resources/11560_0_ROI-of-contracting-excellence.pdf

[2] Agiled. (2026, June 1). *Contract management statistics (2026).* https://agiled.app/statistics/contract-management-statistics